Name			
USN			
Date	Conduction	Record	Viva
Allotted Marks			
Obtained Marks			

EXPERIMENT 8

Implement a word count program in Hadoop and Spark

Aim: To implement a word count program in Hadoop and Spark.

Theory:

Apache Spark is a fast, general-purpose cluster computing system. A **word count** program in Spark demonstrates its ability to process large-scale text data efficiently using **RDD transformations and actions**. The program Counts the frequency of each word in a large text file using distributed computing.

Procedure:

1. **Go to putty , type the user name & password**

2. $ Nano wc.txt
spark is fun
spark is future
Big data is powerfull
Big data is fun

Then press
ctrl x
y
enter
the above line saves the file

```
[maria_dev@sandbox-hdp ~]$ nano wc.txt
[maria_dev@sandbox-hdp ~]$ hdfs dfs -put wc.txt /user/data
```

3. Then type the following
$ hdfs dfs –put wc.txt /user/data

// uploads (copies) a local file named `wc.txt` into the Hadoop Distributed File System (HDFS) at the path `/user/data`.

$ spark-shell

In the terminal it should display
Scala>

4. Once you get scala>
type the following code

scala> val textFile = sc.textFile("hdfs:/user/data/wc.txt")

// This line of code reads the file wc.txt from HDFS into a **Resilient Distributed Dataset (RDD)** called textFile using Apache Spark.

scala> textFile.getClass

// used in Scala to **check the runtime class** of the object textFile.

scala> val words = textFile.flatMap(line => line.split(" "))

// **splits each line of the text file into individual words** and stores them in a new RDD called words

scala> words.collect().foreach(println)

// This line **prints each word** in the words RDD to the console.

scala> val wordPairs = words.map(word => (word, 1))

// This line **prepares each word for counting** by converting it into a key-value pair where the key is the word and the value is 1.

scala> wordPairs.collect().foreach(println)

// This line **prints each (word, 1) pair** from the wordPairs RDD to the console.

scala> val wordCounts = wordPairs.reduceByKey(_ + _)

// This line **aggregates the word counts** — it combines all the (word, 1) pairs by word and **sums their values**, resulting in a count of how many times each word appears.

scala>wordCounts.collect().foreach(println)

// This line **prints the final word counts** (i.e. how many times each word appears) to the console.

```
scala> val textFile = sc.textFile("hdfs:/user/data/wc.txt")
textFile: org.apache.spark.rdd.RDD[String] = hdfs:/user/data/wc.txt MapPartitionsRDD[6] at textFile at <console>:24

scala> textFile.getClass
res4: Class[_ <: org.apache.spark.rdd.RDD[String]] = class org.apache.spark.rdd.MapPartitionsRDD

scala> val words = textFile.flatMap(line => line.split(" "))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[7] at flatMap at <console>:25

scala> words.collect().foreach(println)
spark
is
fun
spark
is
future

scala> val wordPairs = words.map(word => (word, 1))
wordPairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[8] at map at <console>:25

scala> wordPairs.collect().foreach(println)
(spark,1)
(is,1)
(fun,1)
(spark,1)
(is,1)
(future,1)

scala> val wordCounts = wordPairs.reduceByKey(_ + _)
wordCounts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[9] at reduceByKey at <console>:25

scala> wordCounts.collect().foreach(println)
(is,2)
(spark,2)
(future,1)
```

Result:

****